# The Open Source Audit

---

## Cover Page

Project Title: The Open Source Audit

Course Name: Open Source Linux Administration

Your Name: Om Dixit

Registration Number: 24MEI10055

Slot: C11

Date of Submission: 30/03/26

Chosen Software: Git

---

## Introduction

### What is Open Source?

Most of the software that powers the modern world, the servers that run websites, the tools that developers use every day, the operating systems inside supercomputers and smartphones, was not built by a single company and sold for profit. It was built openly, shared freely, and improved collectively. That is the essence of open source.

Open source software is computer software where the source code is made available to everyone that is, anyone can look at the code, change it, run it and share it with others. It is like having the recipe for your favorite dish where you can see all the ingredients and steps, and you can even change the recipe to make it better according to your own taste and liking.

### Why Open Source is Important

Open source software is important because it is usually free to use, which saves money. Secondly many people can work together to make the software better and fix problems quickly. Third, users have the freedom to change the software to meet the needs according to them. Finally, open source software is more secure because many experts can check the code for problems and edit accordingly.

### What This Report Will Do

This report will examine Git, which is a popular open source software tool. We will look at how Git was created, why it was made open source, and how it works on computer systems. We will also compare Git with similar software that is not open source. The goal is to understand the benefits and challenges of open source software through studying Git.

---

# Part A - Origin & Philosophy

## A1. Origin Story

### Who Created Git

Linus Torvalds, the person who created the Linux operating system, created Git in 2005. He is a Finnish software engineer who is known for his work on important open source projects.

### The Problem That Existed

Before Git was created, the Linux kernel development team used a software called BitKeeper to manage their code. BitKeeper was a proprietary software, that is it was not open source. The company that made BitKeeper allowed the Linux team to use it for free, but there were certain restrictions.

In 2005, the relationship between the Linux community and BitKeeper's company became difficult. The company decided to stop providing free access to BitKeeper. This created a huge problem because the Linux team needed a way to manage their code, but they could not pay for proprietary software.

### Why Git Was Created

Linus Torvalds decided to create a new version control system because no existing open source tool could handle the needs of Linux development. The Linux kernel is a huge project with thousands of contributors around the world. The team needed a tool that could:

- Handle big amount of code
- Work fast even with many people making changes
- Work without needing a central server
- Keep track of all changes made to the code

Torvalds wanted a tool that would be fast, simple, and reliable. He started working on Git and completed the basic version in just a few weeks.

### Why Git Was Made Open Source

Git was made open source for many reasons. First, Linus Torvalds believed in the open source philosophy. He had already created Linux as open source software, so it made sense to make Git open source too.

Second, making Git open source meant that the Linux community would never again face the problem they had with BitKeeper. No company could take away their access to the tool or change the rules suddenly.

Third, by making Git open source, Torvalds allowed other developers to improve the software and add new features. This meant Git could become better easily and faster because not only a single company but also other small developers were simultaneously working on to improve the experience of the Git.

## A2. License Analysis

### The Four Freedoms of Open Source

Open source software is based on four basic freedoms:

1. Freedom to Use: Anyone can use the software for any purpose without asking permission or paying money.
2. Freedom to Study: Anyone can look at the source code to understand how the software works.
3. Freedom to Modify: Anyone can change the software to make it work better by adding new features and fixing bugs.
4. Freedom to Share: Anyone can give copies of the software to other people, including versions they have modified without any copyright issue.

### Does Git Give All Four Freedoms?

Git uses the GPL v2 (GNU General Public License version 2) license. This license does provide all four freedoms of open source software.

With Git, you can:

- Use it for any project without paying.
- Read all the source code.
- Make changes to improve it.
- Share your modified version with others.

However, GPL v2 has one very important rule: If you share a modified version of Git, you must also share the source code of your changes. This rule makes sure that improvements to Git remain available to everyone without any issue.

### Can Companies Modify and Sell Git?

Yes, companies can modify Git and sell it, but with some conditions. The GPL v2 license allows this, but any company that sells a modified version of Git must:

- Provide the source code of their changes.
- Allow their customers to modify and share the software.
- Keep the same GPL v2 license on their version.

This means companies cannot take Git, modify it, and then make their version proprietary. The open source nature must be preserved even after taking Git.

### GPL vs MIT Difference

GPL License: called COPYLEFT

- Requires that modified versions remain open source too.

- If you use GPL code in your software, your entire software must be open source as well.
- Protects the freedom of the software strongly.

MIT License: called PERMISSIVE

- Allows people to use the code in proprietary software.
- Does not require that modified versions be open source.
- Gives more freedom to developers but less protection to the software.

Git uses GPL because Linus Torvalds wanted to ensure that Git would always remain free and open source.

### "Free vs Freedom" with Git Example

When people talk about open source software, they often say "free as in speech, not free as in beer." This means:

Free as in Beer (Cost):

- Git costs no money to download and use.
- You does not charge any license fees.
- This is about the price being zero.

Free as in Speech (Freedom):

- Git gives you the freedom to use, study, modify, and share
- You are not restricted in how you use the software
- You have control over the software, not the other way around

Therefore, Git provides both types of freedom, costs nothing in return and allows users to edit with freedom.

## A3. Ethics of Open Source

### Should All Software Be Open Source?

Arguments for making all software open source:

- Everyone could benefit from improvements made by other people.
- Security would be better because more people could find and fix bugs and errors.
- Software would cost much less because companies would not need to spend money protecting their code.
- Innovation would happen very fast because developers could build on each other's work.
- People would have more control over the software they use.

Arguments against making all software open source:

- Companies need to make money from their software to pay developer.

- Some software requires huge investments that might not happen without the promise of profits.
- Competition between companies can drive innovation.
- Some companies might have good reasons to keep their code secret so as to maintain peace.

### Is it Fair for Companies to Use Open Source Software?

Many large companies use open source software like Git to build their products and make money. This raises questions about fairness.

Why it might be fair:

- Companies often contribute back to open source projects by fixing bugs and adding features.
- Open source software was created to be used by everyone.
- Companies provide jobs and services that benefit society.
- Many open source projects are supported by companies that donate money and developer time.

Why it might be unfair:

- Some companies make billions of dollars using open source software but contribute very less back.
- Small open source projects often struggle to get funding while companies profit from their work.
- Companies sometimes take open source projects and create competing proprietary versions from it.

### Impact on Innovation

Open source software like Git has had a huge impact on innovation:

Positive impacts:

- Developers can learn from high-quality code written by experts
- New projects can build on existing open source tools instead of starting from scratch
- Collaboration between developers around the world leads to better solutions
- Standards emerge naturally when everyone can see how things work

Potential negative impacts:

- Companies might invest less in research if they cannot protect their discoveries
- Some types of software might not get developed if there is no way to make money from them

Overall, Git shows how open source can drive innovation. The version control features that Git are now used in almost all software development, and many new tools have been built on top of Git.

---

## Part B - Linux Footprint

### Installation Method

To install Git on a Linux system, the most common method is using the package manager. On Ubuntu and other Debian-based systems, the command is: sudo apt install git

- sudo gives administrator privileges needed to install software
- apt is the package manager for Debian-based Linux systems
- install tells apt to install the software
- git is the name of the package to install

### File Locations

After installation, Git files are stored in several locations:

### Binary Location

The main Git program is installed at:

/usr/bin/git

This directory contains the executable file that runs when you type git in the terminal. The /usr/bin/ directory is where most user programs are stored on Linux systems.

### Configuration Location

User configuration is stored by Git in:

~/.gitconfig

The ~ symbol means the user's home directory. This file contains settings like:

- User name and email address.
- Preferred text editor.
- Color settings for output.
- Custom commands and shortcuts.

### System-wide Configuration

Git also has system-wide settings stored in:

/etc/gitconfig

This file contains settings that apply to all users on the system.

### User and Group Information

Git runs as the current user who executes the command.
This means that:

- If user "alex" runs a git command, Git runs with alex's permissions.
- Git can only access files that the current user can access.
- Git creates new files with the current user as the owner.
- Each user has their own Git configuration settings.

Git does not require a special user account or group to function. This is different from some other software that needs to run as a system user.

### Service Information

Important: Git is NOT a service. This means that:

- Git does not run continuously in the background.
- There is no Git program or script that starts when the computer boots.
- Git only runs when a user himself executes a git command.
- You cannot start, stop, or restart Git like other services as it requires commands for all these operations.

Git is a command-line tool that performs operations when called and then exits. This design makes Git lightweight and simple to use.

### Updates

Git is updated through the same package manager which was used for installation:

```
sudo apt update
sudo apt upgrade git
```

The update process are as follows:

1. `sudo apt update` downloads information about available updates
2. `sudo apt upgrade git` installs the newest version of Git

Users can also update all system packages at once, which will include Git if an update is available.

### Verification

After installation, you can verify Git is working or not with: `git --version`

You can also check the installation location with: `which git`

And view the configuration with: `git config --list`

---

## Part C - The FOSS Ecosystem

### Dependencies

Git relies on several other open source components to function properly. These are:

### C Standard Library

Git is written in the C programming language, so it depends on the C standard library (libc). On most Linux systems, this is provided by glibc (GNU C Library). This library provides basic functions that all C programs need.

### Compression Libraries

Git uses compression to save space when storing code history. It depends on:

- zlib: A library for data compression that makes Git repositories smaller
- libcurl: Used for network operations when Git communicates with remote servers

### Cryptographic Libraries

Git uses cryptographic functions to ensure data integrity:

- OpenSSL or similar: Provides encryption functions for secure communication
- SHA-1 hashing: Built into Git for creating unique identifiers for each change

### Shell and Core Utilities

Git relies on basic Unix tools that are part of every Linux system:

- Shell commands for file operations
- Text processing tools
- Network utilities

### Tools Built on Git

Many popular development tools and platforms are built on top of Git:

### GitHub

GitHub is a web-based platform that uses Git for version control. It adds:

- A web interface for managing Git repositories
- Collaboration tools like issue tracking and pull requests
- Social features that let developers follow each other's work
- Automated testing and deployment tools

GitHub has made Git much more popular because it provides an easy way for people to share and collaborate on code.

### GitLab

GitLab is similar to GitHub but offers some different features:

- Can be installed on private servers
- Includes built-in continuous integration tools
- Provides project management features
- Offers both free and paid versions

### Other Git-based Tools

- Gitea: A lightweight alternative to GitHub that can run on smaller servers
- Bitbucket: Another Git hosting service with good integration with other development tools
- SourceForge: An older code hosting platform that now supports Git
- Git GUIs: Desktop applications that provide graphical interfaces for Git commands

## Community and Governance

### Contributors

Git is maintained by a community of developers around the world:

- Junio Hamano is the current maintainer who oversees Git development
- Hundreds of developers have contributed code to Git
- Anyone can propose changes or report bugs
- New features are discussed openly on mailing lists

### Linux Foundation

The Linux Foundation provides support for Git development:

- Pays for infrastructure like servers and websites
- Helps coordinate between different contributors
- Provides legal support for the project
- Promotes Git adoption in the software industry

### Documentation and Support

The Git community maintains extensive documentation:

- Official documentation on the Git website
- Books like “Pro Git” that are freely available online
- Tutorials and guides for beginners
- Community forums where users help each other

Therefore, this ecosystem shows how open source projects can grow beyond their original purpose. Git started as a tool for Linux development but has now become the foundation for software development across many different industries and platforms.

---

## Part D – Comparison between OPEN SOURCE and PROPRIETRY

### Feature Comparison Table

| Feature | Git (Open Source) | BitKeeper (Proprietary) |
|---|---|---|
| Cost | Free for everyone | Paid licenses required |
| Security | Open audit - anyone can check code for problems | Closed source - only company can audit |
| Support | Community support through forums and documentation | Company provides official support |
| Freedom | Full freedom to use, modify, and share | Limited by license terms |
| Control | Community controls development direction | Company controls all development |
| Source Code | Available to everyone | Hidden from users |
| Customization | Anyone can modify to fit their needs | Only company can make changes |
| Development Speed | Many contributors can work on improvements | Limited to company employees |
| Long-term Availability | Cannot be discontinued by any single entity | Company could stop development |

### Analysis and Conclusion

### Advantages of Git (Open Source)

Git's open source nature provides several important advantages over proprietary alternatives like BitKeeper. The most obvious benefit is cost - Git is completely free to use, while BitKeeper requires paid licenses. This makes Git accessible to individual developers, small companies, and large organizations alike.

Security is another major advantage of Git. Because the source code is available for anyone to examine, security experts around the world can find and fix vulnerabilities quickly. With BitKeeper, users must trust that the company has found and fixed all security problems, but they cannot verify this themselves. The open nature of Git means that security issues are often discovered and patched faster than in proprietary software.

### Advantages of BitKeeper (Proprietary)

BitKeeper does have some advantages, particularly in terms of support. Companies that pay for BitKeeper licenses receive dedicated technical support from the company. If something goes wrong, they have someone to call who is responsible for fixing the problem. With Git, users rely on community support, which can be very helpful but is not guaranteed.

BitKeeper may also have some advanced features that were developed specifically for largescale commercial use. Since the company can invest in research and development without sharing their innovations with competitors, they might be able to create specialized features faster than the open source community.

### Conclusion

After comparing Git and BitKeeper, I believe Git's open source model is clearly superior for most users and situations. The freedom, cost savings, security benefits, and community support far outweigh the advantages of proprietary software like BitKeeper. The fact that Git has become the dominant version control system used by millions of developers worldwide proves that the open source approach works well for this type of software. To be honest, it's hard to even imagine writing code today without it. It completely changed the game for how teams actually collaborate and stay organized. The way it backs the whole open-source movement is probably its biggest win.

The original conflict between the Linux community and BitKeeper that led to Git's creation demonstrates one of the biggest risks of proprietary software - vendor lock-in. When users depend on proprietary software, the vendor has significant power over them and can change terms or discontinue products at any time. With Git, no single entity has this kind of control, making it a more reliable long-term choice.

---

## Shell Scripts Section

### Script 1: System Identity

Purpose: This script displays basic information about the system and Git installation.

```
#!/bin/bash
echo "=== System Identity ===" echo "Current User: $(whoami)" echo
"Hostname: $(hostname)" echo "Operating System: $(uname -s)" echo "Kernel
Version: $(uname -r)" echo "Git Version: $(git --version)" echo "Git Location:
$(which git)" echo "Date: $(date)"
```

Explanation: This script uses several basic shell concepts:

- Variables and command substitution with $(command)
- The echo command to display text
- System commands like whoami, hostname, and uname
- The Git command to check version and location

### Script 2: Git Configuration Check

Purpose: This script checks the current Git configuration settings.

```
#!/bin/bash
echo "=== Git Configuration Check ===" echo "User Name: $(git config --get
user.name)" echo "User Email: $(git config --get user.email)" echo "Default
Editor: $(git config --get core.editor)" echo "Global Config File: ~/.gitconfig"
if [ -f ~/.gitconfig ]; then     echo "Config file exists: Yes"
   echo "Config file size: $(ls -lh ~/.gitconfig | awk '{print $5}')" else
   echo "Config file exists: No" fi
```

Explanation: This script demonstrates:

- Git configuration commands
- Conditional statements with if [ -f filename ]
- File testing to check if the config file exists
- Command piping with ls -lh | awk
- String manipulation and variable expansion

### Script 3: Repository Status Checker

Purpose: This script checks if the current directory is a Git repository and shows its status.

```
#!/bin/bash
echo "=== Repository Status Checker ===" echo "Current
Directory: $(pwd)"

if git rev-parse --git-dir > /dev/null 2>&1; then     echo "This is a Git
repository: Yes"
   echo "Repository root: $(git rev-parse --show-toplevel)"     echo "Current branch: $(git
branch --show-current)"     echo "Number of commits: $(git rev-list --count HEAD)"
echo "Last commit: $(git log -1 --format='%h - %s (%cr)')" else
   echo "This is a Git repository: No"     echo "To create a repository,
run: git init" fi
```

Explanation: This script shows:

- Advanced conditional testing with Git commands
- Error redirection with > /dev/null 2>&1
- Git-specific commands for repository information

- Command formatting with Git log options
- Nested command execution

### Script 4: File System Usage

Purpose: This script shows disk usage and Git-related directory sizes.

```
#!/bin/bash
echo "=== File System Usage ===" echo
"Disk Usage Summary:" df -h | grep -E
'^/dev/' echo ""
echo "Git Binary Size: $(ls -lh /usr/bin/git | awk '{print $5}')" echo "Home Directory Usage:
$(du -sh ~ | awk '{print $1}')"

if [ -d ~/.git ]; then
   echo "Git metadata size: $(du -sh ~/.git | awk '{print $1}')" fi
echo "Current directory files:" ls -la | head -10
```

Explanation: This script uses:

- File system commands like df and du
- Text filtering with grep and pattern matching
- Text processing with awk
- Directory testing with [ -d directory ]
- Command output limiting with head

### Script 5: The Open Source Manifesto Generator

Purpose: This script generates a personalised open source philosophy statement by asking the user three questions interactively, then composing a short paragraph using their answers and saving it to a .txt file.

```
  GNU nano 8.7
# Capture current date and set output filename [cite: 197, 198]
DATE=$(date +'%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

# Compose the paragraph using the variables [cite: 199, 200]
echo "------------------------------------------" > "$OUTPUT"
echo "MY OPEN SOURCE MANIFESTO" >> "$OUTPUT"
echo "Created on: $DATE" >> "$OUTPUT"
echo "------------------------------------------" >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "As a developer, I rely on $TOOL to build and manage my work." >> "$OUTPUT"
echo "To me, the core of open source is $FREEDOM, which allows us to innovate." >> "$OUTPUT"
echo "In the spirit of the community, I pledge to build $BUILD and" >> "$OUTPUT"
echo "share it freely with the world, standing on the shoulders of giants." >> "$OUTPUT"

echo ""
echo "Success! Your personalized manifesto has been saved to: $OUTPUT"
echo "--- PREVIEW OF YOUR MANIFESTO ---"
cat "$OUTPUT"
```

Explanation: This script demonstrates:

- Interactive user input command like **read**
- String concatenation with static text
- Multiple command chaining
- System command integration using **date** and **whoami** commands
- Practical philosophy

Each script builds on shell programming concepts while focusing on Git and system administration tasks. They show how open source tools like Git integrate with the broader Linux ecosystem and can be automated using shell scripts.

---

## Conclusion

### What I Learned

Through this audit of Git, I learned many important things about open source software and how it works in practice. First, I discovered that open source software is not just about being free - it is about freedom. Git gives users the freedom to use, study, modify, and share the software, which is much more valuable than just not having to pay money.

I also learned about the practical benefits of open source development. Git was created because a community needed a tool that existing proprietary software could not provide. By making Git open source, Linus Torvalds ensured that the same problem would never happen again. The community now controls the tool instead of depending on a company that might change its mind.

The technical aspects were interesting too. I learned how Git integrates with the Linux system, where its files are stored, and how it depends on other open source components. This showed me that open source software is really an ecosystem where different projects work together to create powerful tools.

### Importance of Open Source

This study of Git has convinced me that open source software is very important for several reasons. First, it provides freedom and control to users. With proprietary software, users must accept whatever the company decides. With open source software like Git, the community can guide development in directions that benefit everyone.

Second, open source software tends to be more secure and reliable. Because many people can examine the code, problems are found and fixed quickly. Git has been tested by millions of developers around the world, making it very stable and trustworthy.

Third, open source software drives innovation. Git introduced new ideas about distributed version control that have influenced how all software development is done today. These innovations happened because Git was open source and other developers could build on its ideas.

Finally, open source software is more sustainable in the long term. Companies can go out of business or change their priorities, but open source projects like Git continue as long as the community finds them useful.

## My Opinion on Git

I believe Git is an excellent example of how open source software should work. It solved a real problem that the development community faced, and it did so in a way that prevents the problem from happening again. Git has become the standard tool for version control because it is technically excellent and because it gives users the freedom they need.

The success of Git also shows the power of open source development. What started as a personal project by Linus Torvalds has become essential infrastructure for the entire software industry. This happened because the open source model allowed thousands of developers to contribute improvements and because platforms like GitHub made Git accessible to everyone.

Git demonstrates that open source software can compete with and even surpass proprietary alternatives. It is faster, more reliable, and more widely used than the proprietary BitKeeper software that originally inspired its creation.

In conclusion, Git is not just a version control system - it is proof that the open source model can create better software while preserving user freedom. This audit has shown me why open source software is important for developers, companies, and society as a whole.